

Bases de datos no relacionales

Práctica MongoDB

Introducción

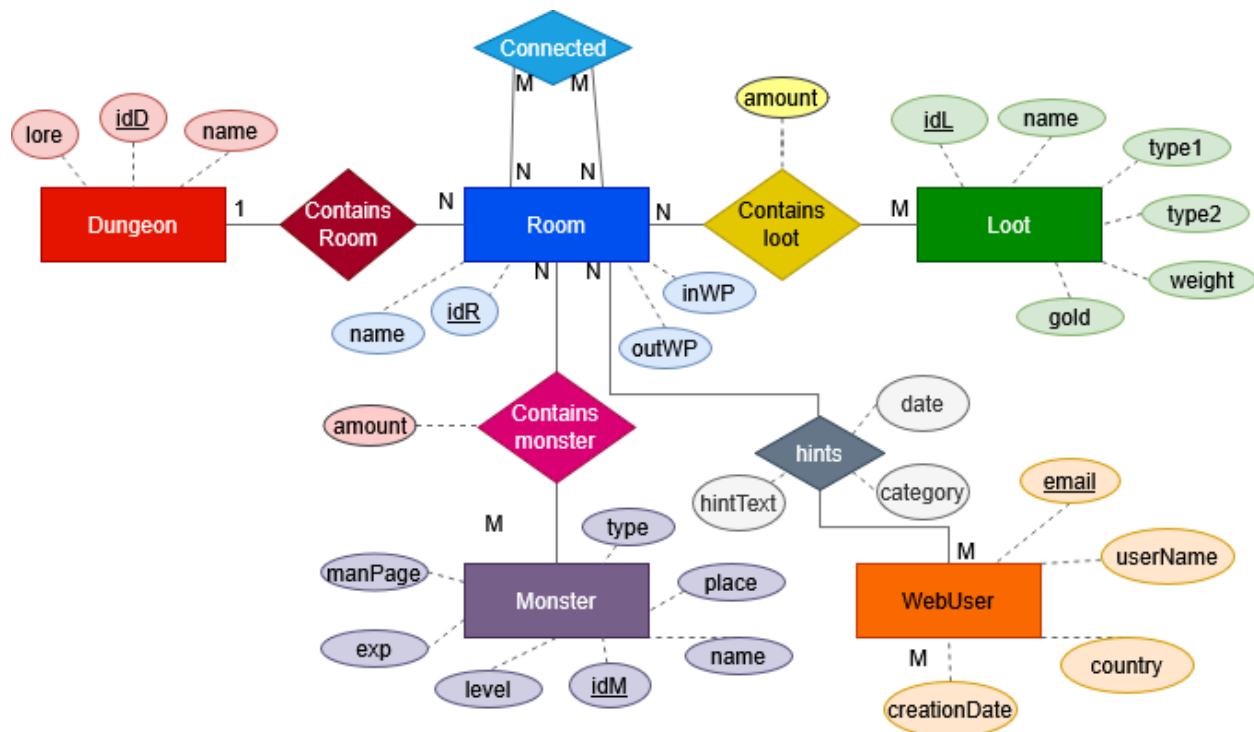
Le empresa de “Norsewind Studios” tiene un MMO muy popular llamado “Jotun’s Lair”, donde los jugadores exploran mazmorras matando monstruos y encontrando loot valioso. El juego lleva funcionando muchos años y el estudio ha desarrollado una wiki que contiene información y lore sobre los monstruos, el loot del juego y las mazmorras. Además, permite a los usuarios dejar comentarios sobre su experiencia con el videojuego. Los usuarios pueden dejar comentarios sobre las siguientes temáticas:

- **Lore:** Teorías e información relacionadas con la historia del juego.
- **Hint:** Trucos y consejos para otros jugadores.
- **Suggestion:** Sugerencias de los usuarios para mejorar algún aspecto del juego.
- **Bug:** Reportes de posibles fallos que ocurren durante el gameplay.

La empresa tiene una API REST para dar servicio a la wiki. Hasta ahora, se viene usando una base de datos relacional, pero últimamente, debido al alto volumen de jugadores, el sistema se está viendo sobre saturado. Además, se está planteando extender esta API para dar servicio a otras aplicaciones internas de la compañía. Los siguientes equipos piden cambios:

- **Quality asurance:** les interesa que se les notifique automáticamente cuando un usuario deje un comentario de tipo Bug/Suggestion.
- **PR:** Los jugadores llevan tiempo pidiendo un tablón donde aparezcan todos los comentarios relativos al lore del juego separado por idiomas.
- **Marketing:** está interesado en conocer información de los jugadores como el promedio de veces que postean al mes, la longevidad de los usuarios activos, los países donde el juego se juega más ... etc.

Por esa razón, se ha decidido migrar de una base de datos relacional a una base de datos de documentos para probar si así se solventan los problemas. A continuación, se muestran las tablas que se van a migrar



Bases de datos relacional que da servicio a la wiki del juego. La parte coloreada es la parte que usaremos en esta práctica.

Objetivo

Se desea hacer un prototipo como prueba de concepto antes de decidirse a realizar la migración. Para el prototipo se usará MongoDB. La API REST actual tiene varios endpoints de los que destacan:

Endpoints

Información de un usuario

GET /user/{email}

Este endpoint recibe el email de un usuario y devuelve todos los campos de un usuario. Además, incluye los 20 últimos comentarios que ha realizado ese usuario. De cada comentario incluye, el texto, la fecha de creación, la categoría, el id (Room.IdR) y nombre (Room.name) de la habitación a la que hace referencia el comentario, el id (Dungeon.IdD) y nombre (Dungeon.name) de la mazmorra donde está la habitación.

Información de habitación en particular

GET /room/{room_id}

Este endpoint recibe el id de una habitación y devuelve la siguiente información de una habitación: *idR*, *name*, *inWP* y *outWP*. Además, incluye el número de monstruos de cada tipo que hay en la habitación y el total de oro que valen los tesoros en la sala. Por último, incluye los últimos 20 comentarios que se hayan realizado para esa habitación, cada comentario debe incluir: *userName*, *country*, *creationDate* del usuario que lo realizó y *texto*, *fecha de publicación* y *categoría* del comentario.

Información de una mazmorra

GET /dungeon/{dungeon_id}

Este endpoint recibe el id de una mazmorra y devuelve información sobre una mazmorra del juego. Debe devolver: *idM*, *name* y *lore*. Además, este endpoint se utiliza para alimentar un grafo interactivo por lo que requiere la siguiente información: 1) el nombre e id de cada habitación de la mazmorra; 2) las conexiones entre habitaciones de la mazmorra; 3) el id y el nombre de los monstruos que aparecen en cada habitación; 4) el id y el nombre de los tesoros que aparecen en cada habitación; 5) El número de comentarios de cada categoría que hay en cada habitación.

Publicar comentario

POST /comment/

Este endpoint añade un nuevo comentario. Recibe como parámetros: *user_email* (str), *room_id* (int), *text* (str), *category* (str).

Borrar monstruo

DELETE /monsters/{monster_id}

Este endpoint recibe el id de un monstruo y lo elimina de la base de datos.

Diseño

Para llevar a cabo el prototipo se ha preparado una base de datos MongoDB con las siguientes colecciones.

Rooms	Users	Loot
room_id: int, room_name: str, dungeon_id: int, dungeon name: str, ?in_waypoint: str, ?out_waypoint: str, rooms_connected: [{ room_id: int, room_name: str }] hints: [{ creation_date: str, text: str, category: str, publish_by: { email: str, user_name: str, creation_date: str, country: str } },...], monsters: [{ id: int, name: str, place: str, type: str, man_page: str, level:int, exp: float },...], Loot: [{ id: int, name: str, type1: str, type2: str, weight: str, gold: int },...]	email: str, user_name: str, creation_date: str, country: str, hints: [{ creation_date: str, text: str, category: str, references_room: { room_id: int, room_Name: str, dungeon_id: int } },...]	id: int, name: str, type1: str, type2: str, weight: str, gold: int, in_rooms: [{ room_id: int, room_name: str, dungeon_id: int dungeon_name: str, },...]
	Monsters	
	id: int, name: str, place: str, type: str, man_page: str, level:int, exp: float, in_rooms: [{ amount: int, room_id: int, room_name: str, dungeon_id: int dungeon_name: str, } },...]	

Tareas

1-Implementar los endpoints

Haz una función python para cada uno de los endpoints del API REST: GET /user/{email}, GET /room/{room_id}, GET /dungeon/{dungeon_id}, POST /comment/, DELETE /monster/{monster_id}. Las funciones se deben conectar a la base de datos MongoDB y realizar las consultas pertinentes. Se deben hacer todas las operaciones que se puedan en la base de datos, no ejecutar realizar cálculos en Python si no es necesario.

2-Consultas analíticas

Usa mongoDB Compass para realiza las siguientes consultas analíticas.

1. El *idM* y *name* de los monstruos que aparecen en las 10 habitaciones con mas comentarios de tipo "Bug". Los monstruos no se deben repetir. Todos los datos se deben devolver en una única consulta.
2. El *dungeon_name* de aquellas mazmorras que han recibido un número de comentarios de tipo "Hint" por encima de la media de todas las mazmorras del juego. Todos los datos se deben devolver en una única consulta.
3. Para cada mazmorra del juego, devolver: el numero de comentarios de cada tipo que se han realizado, el oro total que se puede ganar cogiendo todos los tesoros de la mazmorra, el nivel mediano (percentil 50) de los monstruos de la mazmorra, el número de usuarios de cada país que han realizado comentarios en salas de esa mazmorra. Todos los datos se deben devolver en una única consulta.
4. Para cada tipo de monstruo (campo *type*), devolver un array con todas las mazmorras (campo *dungeon_name*) en las que aparece ese tipo. Cada mazmorra solo debe aparecer una vez en la lista. Todos los datos se deben devolver en la misma consulta.
5. Para cada mazmorra, mostrar el tipo del monstruo (campo *type*) más común que aparece en esa mazmorra. En caso de empate, se deberán mostrar uno los tipos empatados, no importa cuál. Todos los datos se deben devolver en la misma consulta.
6. Mostrar todos los comentarios realizados en la/las habitaciones donde este el encuentro (grupo de monstruos) de mayor nivel medio (campo *level*) de todo el juego. Si hay varias habitaciones que empaten en el nivel medio de su encuentro, se tienen que mostrar todas las habitaciones. Todos los datos se deben devolver en la misma consulta.
7. La ratio oro-nivel de una habitación se calcula dividiendo el total de oro que valen todos los tesoros de una habitación entre el nivel medio de los monstruos que se encuentran en la misma. Se pide realizar una consulta que devuelve aquellas habitaciones que se pueden considerar outliers en la distribución de ratio oro-nivel de todas las habitaciones del juego. Por norma general, se considera que un outlier leve es un valor q que cumpla i o ii (donde Q_1 y Q_3 son el percentil 25 y 75 respectivamente). Todos los datos se deben devolver en la misma consulta:
 - i. $q < Q_1 - 1.5 * IQR$
 - ii. $q > Q_3 + 1.5 * IQR$
 - iii. $IQR = (Q_3 - Q_1)$

3- Patrones de diseño

Desde el equipo de operaciones nos han pasado los siguientes datos sobre el uso de los endpoints

tipo	endpoint	frecuencia
lectura	GET /email/{user_id}	300K al día
lectura	GET /room/{room_id}	500K al día
lectura	GET /dungeon/{dungeon_id}	1M al día
escritura	POST /comment	10K al día
escritura	DELETE /monster/{monster_id}	100 al año

Teniendo en cuenta estos datos de usos ¿Qué patrones de diseño de los vistos en clase se pueden utilizar para mejorar el diseño propuesto?

4- Cambio de esquema

Para dar servicio a los jugadores y el equipo de quality assurance se ha decidido crear una colección nueva en la base de datos que contenga todos los comentarios. Para ello sigue estos pasos:

- A. Haz una consulta que obtengan los datos necesarios para la colección y exporta el resultado a un fichero .json. Debajo puedes encontrar la estructura de la colección.
- B. A continuación, crea una nueva colección llamada Hints y usa el fichero .json para poblarla. Además, elimina el campo *hints* de las colecciones *Rooms* y *User*.
- C. Por último, actualiza las funciones de los endpoints: **POST /comment**, **GET /dungeon/{dungeon_id}**, **GET /room/{room_id}** y **GET /user/{email}**. ¿Como se ven afectados estos endpoints?

Hints
<pre>Creation_date: str, HintText: str, Category: str, References_room: { IdR: int, Name: str, IdD: int Dungeon: str, } Publish_by: { Email: str, User_name: str, CreationDate: str, Country: str }</pre>